

GUÍA 1 — Handoff técnico para continuar PielCalma

1. Objetivo de esta guía

Esta guía explica cómo recibir, levantar, revisar y mejorar el proyecto **PielCalma**, un MVP desarrollado para la Hackathon Desafío IA Bagó Perú.

El objetivo no es rehacer la app desde cero. El objetivo es tomar una base funcional ya desarrollada y mejorarla técnicamente sin romper la demo estable.

PielCalma ya cuenta con:

- landing principal;
- Modo Calma;
- Registro inteligente;
- Observador Visual;
- Reporte Médico;
- APIs internas simuladas;
- diseño visual con identidad propia;
- narrativa de producto;
- límites éticos implementados en la interfaz.

La prioridad técnica es fortalecer el MVP sin comprometer la estabilidad.

2. Qué recibirás

Deberías recibir una carpeta comprimida llamada algo como:

```
pielcalma.zip
```

Dentro debe estar el proyecto Next.js sin las carpetas pesadas `node_modules` y `.next`.

La estructura esperada debe incluir archivos como:

```
pielcalma/  
  public/  
  src/  
  .gitignore  
  AGENTS.md  
  CLAUDE.md  
  eslint.config.mjs  
  next.config.ts  
  next-env.d.ts
```

```
package.json
package-lock.json
postcss.config.mjs
README.md
tsconfig.json
```

Importante:

- Si no viene `node_modules`, está bien.
- Si no viene `.next`, está bien.
- Esas carpetas se regeneran localmente.

3. Qué instalar antes de abrir el proyecto

Instalar:

1. **Node.js LTS**
2. **Git**
3. **Cursor** o **Visual Studio Code**
4. **Google Chrome**
5. Cuenta de **GitHub** o acceso al repositorio oficial de la hackathon
6. Opcional: cuenta de **Vercel** para deploy rápido
7. Opcional: acceso a **Azure OpenAI**, **OpenAI API** o el proveedor de IA que entregue la hackathon
8. Opcional: cuenta de **Supabase** si se decide persistir datos

Versiones recomendadas:

```
Node.js: versión LTS actual
npm: viene incluido con Node.js
Editor: Cursor o VS Code
```

Para verificar Node y npm:

```
node -v
npm -v
```

Si ambos comandos muestran números de versión, está bien instalado.

4. Cómo abrir el proyecto

Descomprimir el ZIP.

Abrir la carpeta `pielcalma` en Cursor o VS Code.

Desde la terminal integrada del editor, ubicarse dentro de la carpeta del proyecto.

Ejemplo en Windows:

```
cd "C:\Users\User\Desktop\Hackathon Bago\pielcalma"
```

Luego instalar dependencias:

```
npm install
```

Después levantar el servidor:

```
npm run dev
```

Abrir en el navegador la URL que indique la terminal.

Normalmente será:

```
http://localhost:3000
```

Si el puerto 3000 está ocupado, Next.js puede usar otro puerto, por ejemplo:

```
http://localhost:3001
```

Usar siempre el puerto que indique la terminal.

5. Rutas principales que deben funcionar

Probar estas rutas:

/	Landing principal
/calma	Modo Calma
/registro	Registro inteligente
/observador	Observador Visual
/reporte	Reporte Médico

Ejemplo:

```
http://localhost:3000  
http://localhost:3000/calma  
http://localhost:3000/registro
```

```
http://localhost:3000/observador  
http://localhost:3000/reporte
```

Si el puerto es 3001, usar:

```
http://localhost:3001
```

6. APIs internas simuladas

También deben funcionar estas rutas API:

```
/api/calm-summary  
/api/visual-observation  
/api/report
```

Probar en navegador:

```
http://localhost:3000/api/calm-summary  
http://localhost:3000/api/visual-observation  
http://localhost:3000/api/report
```

Deben devolver JSON.

Estas APIs están simuladas para que la app funcione aunque todavía no haya IA real conectada.

La lógica actual es:

```
/calma      -> llama a /api/calm-summary  
/registro   -> llama a /api/calm-summary  
/observador -> llama a /api/visual-observation  
/reporte    -> llama a /api/report
```

7. Archivos clave del proyecto

Páginas principales:

```
src/app/page.js  
src/app/calma/page.js  
src/app/registro/page.js
```

```
src/app/observador/page.js
src/app/reporte/page.js
```

APIs internas:

```
src/app/api/calm-summary/route.js
src/app/api/visual-observation/route.js
src/app/api/report/route.js
```

Componentes de identidad visual:

```
src/components/BrandNav.js
src/components/SafeDisclaimer.js
```

Archivos globales:

```
src/app/layout.js
src/app/globals.css
```

Dependencias principales:

```
next
react
react-dom
tailwind
recharts
lucide-react
jspdf
html2canvas
clsx
```

8. Primer paso antes de modificar: crear respaldo

Antes de tocar nada, crear una versión estable.

Opción A: copiar carpeta manualmente.

Crear una copia de la carpeta:

```
pielcalma_backup_estable
```

Opción B: usar Git.

Dentro del proyecto:

```
git init
git add .
git commit -m "Version estable inicial de PielCalma"
```

Después, crear una rama para mejoras:

```
git checkout -b mejoras-tecnicas
```

Regla importante:

No modificar directamente la versión estable sin tener backup.

9. Checklist inicial de verificación

Antes de desarrollar, verificar:

```
[ ] npm install termina correctamente.
[ ] npm run dev levanta el proyecto.
[ ] Landing carga.
[ ] /calma carga.
[ ] /registro carga.
[ ] /observador carga.
[ ] /reporte carga.
[ ] /api/calm-summary devuelve JSON.
[ ] /api/visual-observation devuelve JSON.
[ ] /api/report devuelve JSON.
[ ] No hay errores graves en terminal.
[ ] No hay errores graves en consola del navegador.
[ ] El diseño se ve coherente.
[ ] El reporte se puede imprimir.
```

Para abrir consola del navegador:

Chrome -> clic derecho -> Inspeccionar -> Console

10. Flujo demo que debe mantenerse intacto

No romper este flujo:

1. Landing
2. Modo Calma
3. Registro inteligente
4. Observador Visual
5. Reporte Médico

Historia de demo:

Ana, madre de Lucas, entra preocupada porque su hijo no durmió bien por la comezón.
La app no diagnostica.
Primero la ayuda a respirar y ordenar lo observado.
Luego Ana registra sueño, comezón, zonas y posibles factores.
Después sube una foto para generar una observación visual descriptiva.
Finalmente, PielCalma genera un reporte claro para el dermatólogo.

Este flujo debe durar máximo 2 minutos en el pitch.

11. Prioridades técnicas recomendadas

Prioridad 1 — Estabilidad

Antes de agregar IA real o Supabase:

- ☐ Revisar rutas.
- ☐ Revisar APIs.
- ☐ Revisar responsive.
- ☐ Revisar reporte.
- ☐ Revisar impresión.
- ☐ Revisar que no haya lenguaje médico riesgoso.

No agregar nuevas funcionalidades si el flujo base todavía no está estable.

Prioridad 2 — Deploy

Subir el proyecto a una URL pública.

Opción recomendada:

Vercel

Pasos sugeridos:

1. Subir proyecto a GitHub o repositorio oficial.
2. Crear proyecto en Vercel.
3. Conectar repositorio.
4. Deploy automático.
5. Probar rutas públicas.

Comandos útiles antes de deploy:

```
npm run build
```

Si `npm run build` falla, corregir antes de desplegar.

Prioridad 3 — Conectar IA real en texto

Conectar IA real primero solo en:

```
/api/calm-summary  
/api/report
```

No empezar por visión. Texto es más seguro, rápido y útil para demo.

Variables de entorno posibles:

Para OpenAI:

```
OPENAI_API_KEY=...
```

Para Azure OpenAI:

```
AZURE_OPENAI_ENDPOINT=...  
AZURE_OPENAI_API_KEY=...  
AZURE_OPENAI_DEPLOYMENT=...  
AZURE_OPENAI_API_VERSION=...
```

Importante:

```
Nunca poner una API key secreta en frontend.  
Nunca usar NEXT_PUBLIC_OPENAI_API_KEY.  
Las claves secretas deben usarse solo en API routes del backend.
```

La respuesta de IA debe ser JSON estructurado para no romper el frontend.

Formato esperado para `/api/calm-summary`:

```
{
  "mensajeEmpatico": "string",
  "datosOrdenados": ["string", "string", "string"],
  "posibleCoincidencia": "string",
  "preguntaDermatologo": "string",
  "disclaimer": "string"
}
```

Formato esperado para `/api/report`:

```
{
  "resumenSemanal": "string",
  "patronesObservados": ["string"],
  "preguntasDermatologo": ["string"],
  "observacionesVisuales": ["string"],
  "calmaFamiliar": 72,
  "disclaimer": "string"
}
```

Prioridad 4 — Mejorar Observador Visual

El Observador Visual actual funciona con API simulada.

Se puede mejorar de tres formas:

Opción A: mantenerlo simulado

Más estable para demo. Recomendado si queda poco tiempo.

Opción B: IA multimodal

Enviar imagen a un modelo con visión y pedir una descripción estrictamente no diagnóstica.

Debe devolver:

```
{
  "observacionVisual": "string",
  "comparacionAnterior": "string",
  "indiceVisualCambio": "string",
  "limitaciones": "string",
  "disclaimer": "string"
}
```

Opción C: OpenCV básico

Calcular un índice visual simple, por ejemplo:

Índice Visual de Cambio

Pero debe explicarse como:

Indicador comparativo y descriptivo, no clínico.

No debe llamarse:

severidad
nivel clínico
riesgo médico
diagnóstico visual

Prioridad 5 — Supabase

Conectar Supabase solo si hay tiempo.

Posibles tablas:

caregivers
children_profiles
flare_logs
skin_images
medical_reports

Campos sugeridos:

```
caregivers:
- id
- name
- email
- created_at

children_profiles:
- id
- caregiver_id
- name
- age
- condition_label
- created_at
```

```
flare_logs:
- id
- child_id
- log_date
- itch_level
- sleep_quality
- affected_areas
- possible_triggers
- routine_status
- caregiver_emotion
- notes
- created_at

skin_images:
- id
- flare_log_id
- image_url
- visual_observation
- comparison_summary
- visual_change_index
- created_at

medical_reports:
- id
- child_id
- start_date
- end_date
- ai_summary
- observed_patterns
- doctor_questions
- created_at
```

Variables de entorno Supabase:

```
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
```

La conexión puede hacerse desde:

```
src/lib/supabaseClient.js
```

Pero si no hay tiempo, mantener datos demo locales.

12. Reglas críticas de seguridad médica

La app nunca debe decir:

Esto es dermatitis severa.
Esto parece infección.
Debes aplicar X crema.
Debes cambiar el tratamiento.
No necesitas ir al médico.
Esto es grave.
Esto es leve.
El tratamiento no está funcionando.

La app sí puede decir:

Se observa enrojecimiento visible.
Se observa resequedad aparente.
Se observa una posible coincidencia con calor y sudor.
Esto no confirma una causa médica.
Puede conversarse con el dermatólogo.
Esta observación no constituye diagnóstico médico.

Palabras a evitar:

diagnóstico
severidad
leve
moderado
severo
tratamiento recomendado
medicación
dosis
infección
urgencia médica

Salvo para decir explícitamente que la app no hace eso.

13. Prompt base para IA real

Usar este prompt como sistema o instrucción principal:

Eres PielCalma, un asistente de acompañamiento para cuidadores de niños con dermatitis atópica.

Tu función es ayudar a ordenar observaciones cotidianas, reducir carga mental y preparar información útil para conversar con el dermatólogo.

Reglas obligatorias:

- No diagnostiques.

- No clasifiques severidad clínica.
- No uses términos como leve, moderado o severo.
- No recomiendes medicamentos, cremas, dosis ni cambios de tratamiento.
- No digas que el usuario debe o no debe ir al médico.
- No reemplaces al dermatólogo.
- No afirmes causalidad médica.
- Usa lenguaje empático, claro y prudente.
- Usa frases como “se observa”, “posible coincidencia”, “podría ser útil registrar”, “puede conversarse con el dermatólogo”.
- Siempre incluye un disclaimer breve.

Devuelve siempre JSON válido.

14. Tareas concretas para el perfil técnico

Trabajar en este orden:

Fase 1 — Setup y backup

- [] Descomprimir proyecto.
- [] npm install.
- [] npm run dev.
- [] Probar rutas.
- [] Crear commit estable.

Fase 2 — Deploy

- [] Subir a GitHub o repo oficial.
- [] Ejecutar npm run build.
- [] Corregir errores si aparecen.
- [] Deploy en Vercel.
- [] Probar URL pública.

Fase 3 — IA texto

- [] Crear variables de entorno.
- [] Conectar /api/calm-summary con IA real.
- [] Conectar /api/report con IA real.
- [] Forzar salida JSON.
- [] Validar que no diagnostique.
- [] Tener fallback simulado si falla la IA.

Fase 4 — Observador Visual

- ☐ Decidir si será simulado o IA multimodal.
- ☐ Si se conecta IA visual, reforzar prompt ético.
- ☐ Nunca usar clasificación médica.
- ☐ Mantener el output actual del endpoint.

Fase 5 — Supabase opcional

- ☐ Crear proyecto Supabase.
- ☐ Crear tablas.
- ☐ Conectar registros.
- ☐ Conectar imágenes si hay tiempo.
- ☐ No romper demo si falla la base de datos.

Fase 6 — QA final

- ☐ Probar flujo completo.
- ☐ Probar responsive.
- ☐ Probar impresión.
- ☐ Probar deploy.
- ☐ Revisar consola.
- ☐ Revisar lenguaje médico.
- ☐ Preparar demo de respaldo.

15. Qué NO hacer

No hacer esto:

- Rehacer toda la app desde cero.
- Cambiar la narrativa principal.
- Convertirla en chatbot médico.
- Agregar diagnóstico por imagen.
- Agregar recomendaciones de tratamiento.
- Entrenar modelo propio desde cero.
- Implementar login complejo si no aporta al demo.
- Meter demasiadas funcionalidades y romper la estabilidad.

La app ya tiene una base de producto. El trabajo técnico debe fortalecerla, no desviarla.

16. Entregables técnicos ideales

Al final, entregar:

- [] Repositorio actualizado.
- [] URL pública funcionando.
- [] README actualizado.
- [] Variables de entorno documentadas.
- [] APIs funcionando.
- [] Demo estable.
- [] Explicación breve de IA usada.
- [] Capturas del flujo.
- [] Fallback si la IA falla.

17. Frase técnica para explicar al jurado

PielCalma usa IA generativa en tres puntos: primero, para adaptar el acompañamiento emocional en Modo Calma; segundo, para convertir registros diarios en resúmenes y posibles coincidencias no diagnósticas; tercero, para generar reportes claros con preguntas útiles para el dermatólogo. En el módulo visual, la IA se limita a observaciones descriptivas y comparativas, sin diagnóstico ni clasificación clínica.

18. Prioridad final

Si el tiempo es limitado, priorizar así:

1. Demo estable.
2. Deploy.
3. IA real en texto.
4. Observador visual seguro.
5. Supabase.

La demo estable vale más que una arquitectura avanzada que pueda fallar.